UNIVERSITY OF GHANA

DEPARTMENT OF COMPUTER SCIENCE

DCIT 204/308: Data Structures and Algorithms I and II

Joint DSA Semester Project

Ghana Smart Service Operations Optimizer

University of Ghana Hospital, Legon: Operations Model

Local context: University of Ghana Hospital, Legon, a 140 bed hospital. The system handles triage, dispatch, routing, referrals and budget limited resourcing.

Team: Group 41, The Charismatic Enigmas

22248029 22040255 22104879 11102882 22224236 22015166 22160537 22399601 22392509 22372680
22372639 22395395 22367946 22305896 22370298 22399010

There are 16 team members. Each one is identified by their university index number. The full list of who worked on what is in Section 11.

Link to Presentation Video: https://youtu.be/urI4QPQ3M0Y

2. What the project is about, what we assumed, and what the system does and does not do

What the problem is

The University of Ghana Hospital, Legon is a 140 bed teaching hospital. Every day, staff have to make decisions like: which job to handle next (a patient transfer, a pharmacy delivery, an outpatient visit), which porter to send, what is the fastest way to get from one point on campus to another, which places can actually be reached from a given point, and how to fit as many jobs as possible into a limited number of porter minutes. This project is a console program that answers these questions. It uses data structures and algorithms that we built ourselves, and it stores its data in a real MySQL database. The data is modelled on the real hospital, but built by us.

This project is not about building a nice looking screen. The marks are for building the data structures and algorithms correctly, testing them, measuring how fast they are, and connecting everything to a real database.

What we assumed

- The system only models the University of Ghana Hospital, Legon itself. It does not model the University of Ghana Medical Centre, which is a separate and much bigger hospital nearby.

- Each road already has one combined weight (called weighted_cost_min) that mixes distance and road condition together. This means our routing algorithms only have to compare one number, not two.
- For the budget style job selection, porter minutes are the only limit we model. We do not model staff numbers or equipment.
- All our timing tests were run with the same Java settings (a fixed memory size, using -Xms2g Xmx2g) on one laptop. This means the numbers can be fairly compared to each other, but not to a different computer.
- The service request data used to test the system is made up by us, following real patterns, not copied from real patient records.

What goes in and what comes out (5 main jobs the system does)

Job	**What goes in and what comes out**
Pick the next job to handle	In: the current list of waiting jobs, sorted by urgency and deadline. Out: the one job that should be handled next, removed from the list.
Find the shortest route	In: a start location, an end location, and the map of roads. Out: the list of locations to pass through to get there, and the total time in minutes. If there is no way to get there, the list is empty.
Find out what can be reached	In: a starting location and the map of roads. Out: every location that can be reached from that point, and how many steps away each one is.
Pick jobs to fit a budget	In: a list of jobs, each with a cost in porter minutes and a value, plus a total budget in minutes. Out: the group of jobs that gives the most value without going over budget.
Save and load data	In: CSV files or data already in memory. Out: rows saved in the six MySQL tables. Later, the same rows are read back and rebuilt into our own data structures.

What the system does not do

This is a single user console program. One person runs it at a time. It does not watch patients in real time, it does not connect to hospital equipment, and it does not have logins for many users at once. Everything an examiner needs to see can be reached from the menu on screen. Nobody needs to open or change the code while it is running.

3. The data we used, and the database

Where the data came from

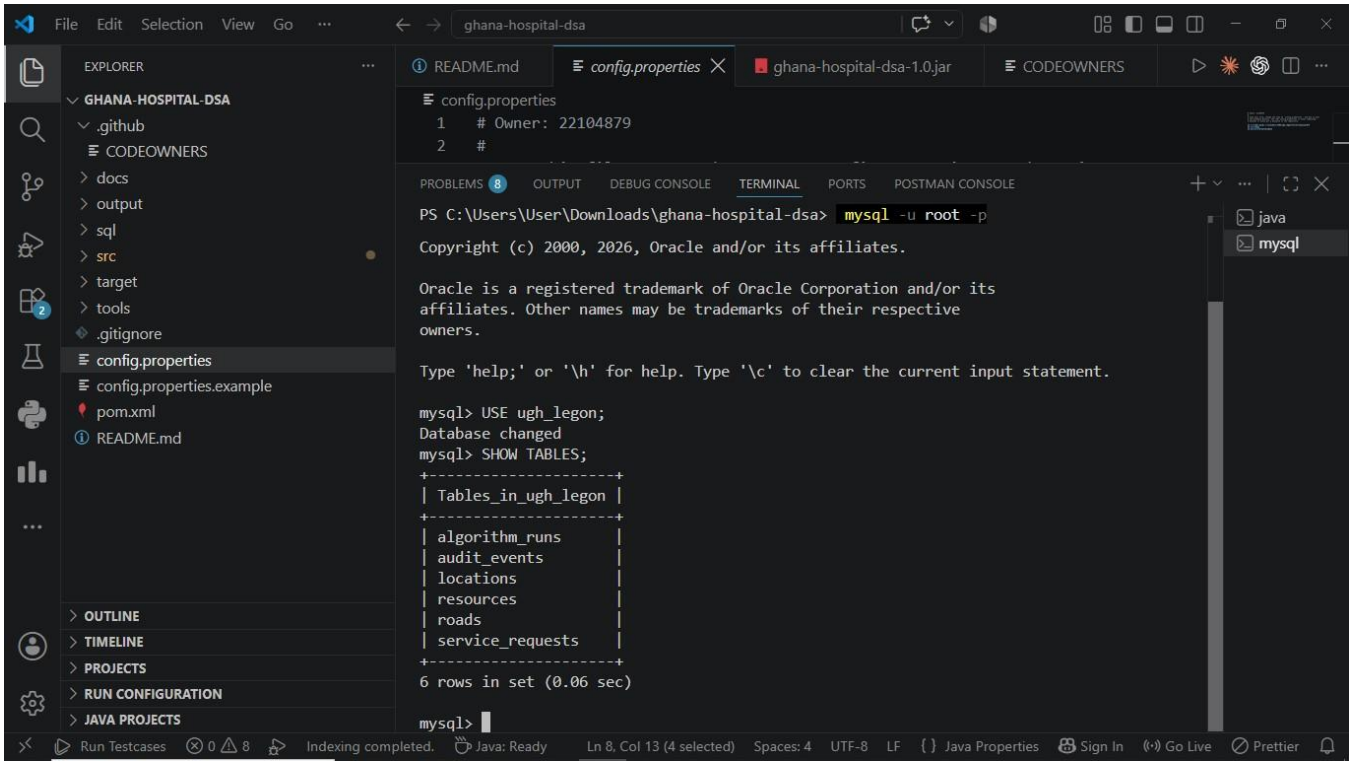
The map coordinates for each location came from looking at the University of Ghana Hospital, Legon compound on Google Earth. Distances and travel times between locations were worked out from the same images, together with normal walking speed. The service request records (the actual jobs, like patient transfers or pharmacy runs) were generated by a small program we wrote (tools/RequestGenerator.java). They follow real hospital categories, such as outpatient visits, dermatology, and pharmacy dispatch, and they use real triage colour codes. They are not taken from any real patient. No private information is used anywhere in this project.

What is in each table

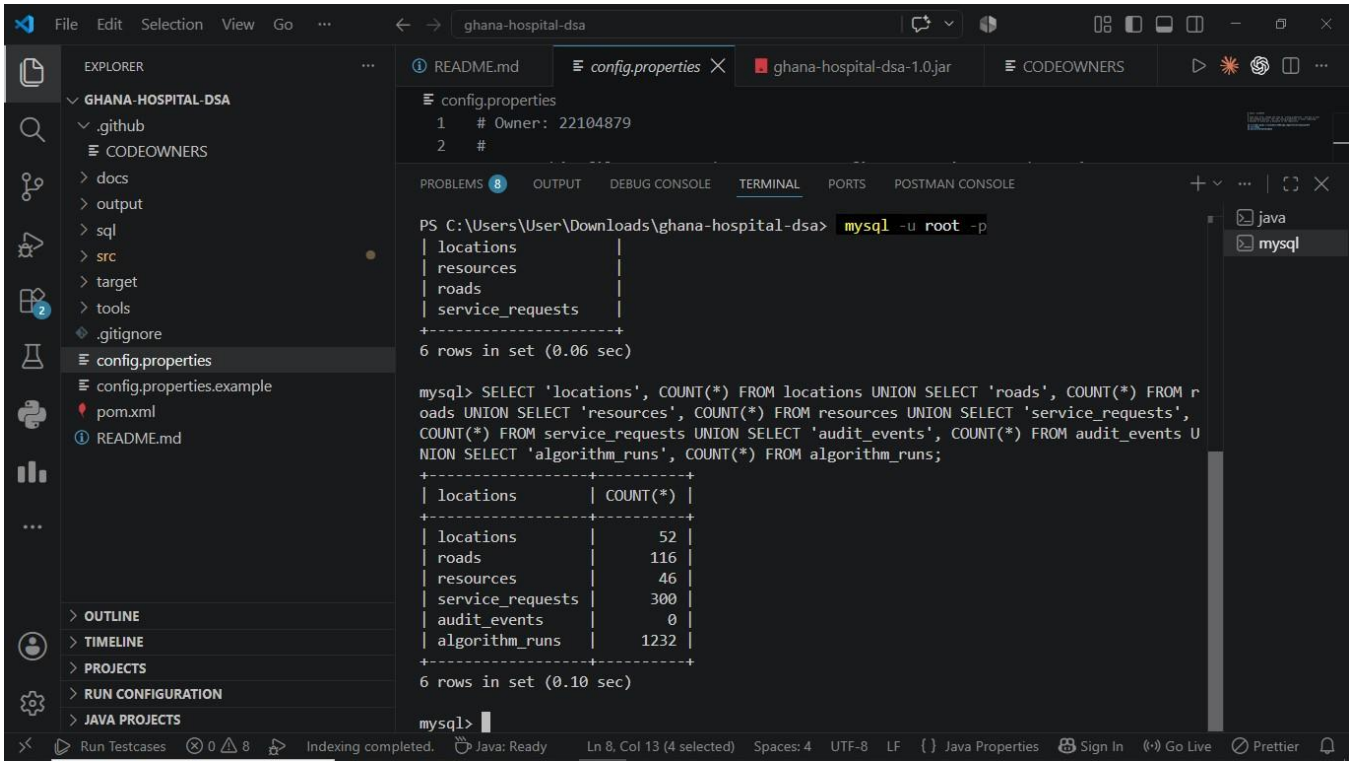
Table	What it stores	How many rows
locations	Every place in the hospital network, like wards and entrances	52
roads	The connections between locations, with distance, time, and condition	116
service_requests	The jobs that need to be queued, prioritised, and searched	300
resources	The porters, vehicles, and staff who can be given jobs	46
algorithm_runs	Timing results from our speed tests	1232, growing every time a test is run
audit_events	A log of undo actions and other important events	filled in when the undo menu is used

Proof the database is real and working

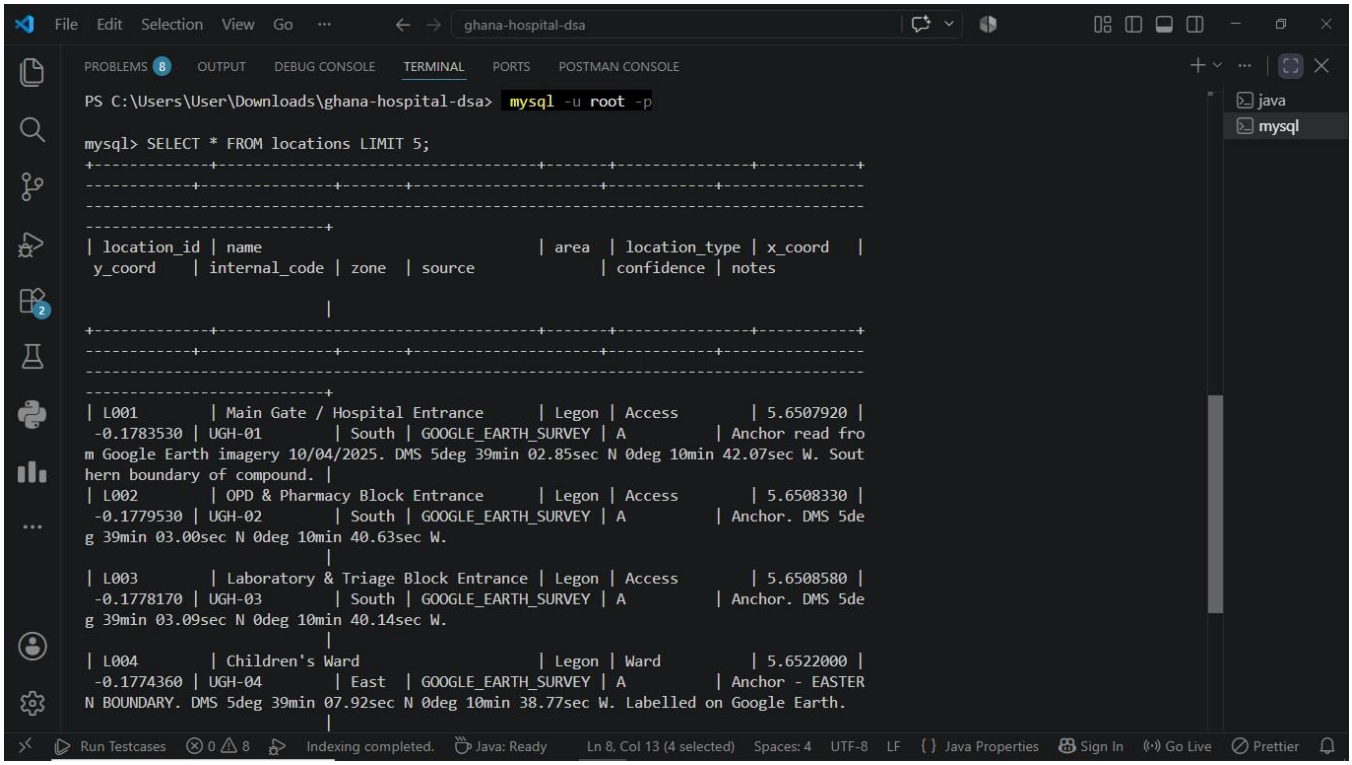
The pictures below were taken straight from a MySQL command line window, connected to the ugh_legon database. They show the data is really stored in MySQL, not just kept inside the Java program while it runs.



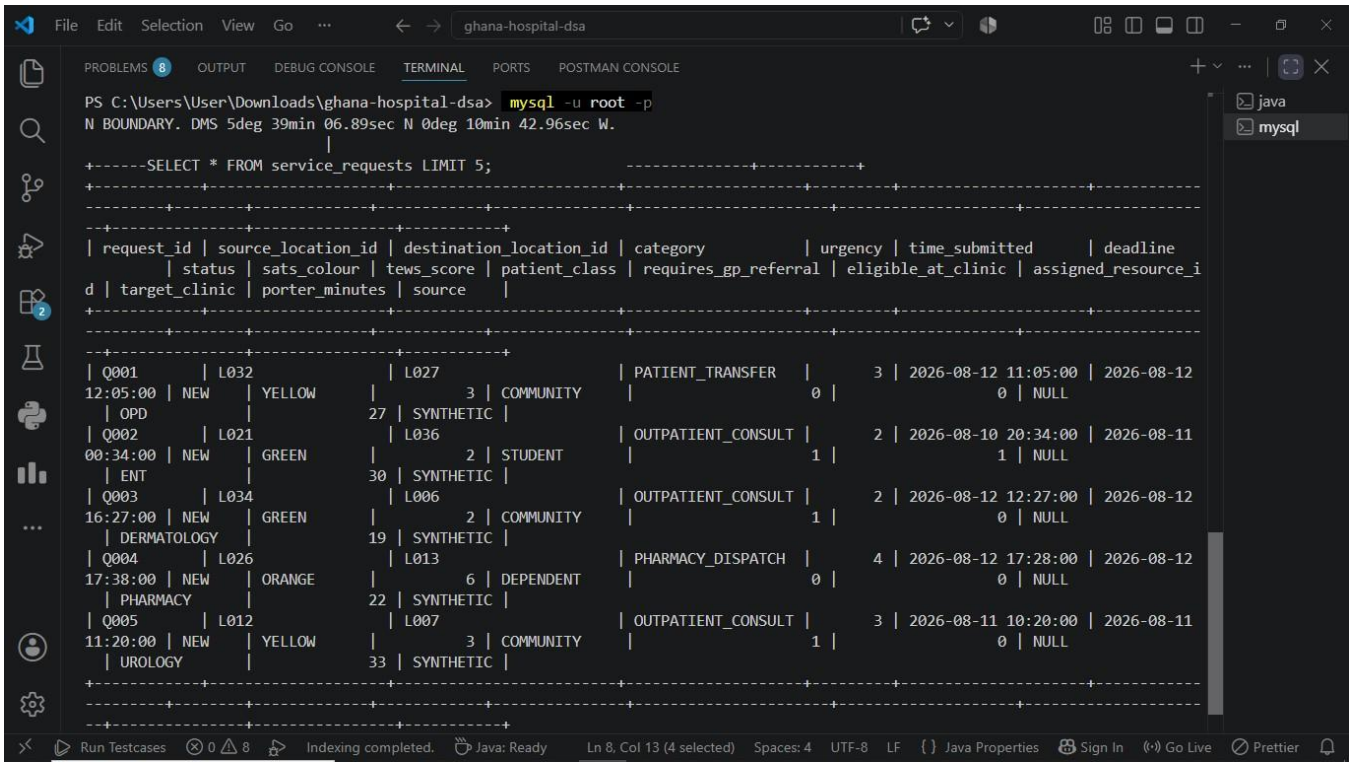
Picture 3.1: the SHOW TABLES command lists all six required tables inside ugh_legon.



Picture 3.2: the row counts for every table. locations 52, roads 116, resources 46, service_requests 300, algorithm_runs 1232, audit_events 0 (this is because the undo menu had not been used yet in this session).



Picture 3.3: real rows from the locations table, including GPS coordinates and notes on where each one came from.



Picture 3.4: real rows from the service_requests table, showing urgency, triage colour, patient type, and deadline.

4. How the system is built

The program is written in Java 21 and built with Maven. It runs as a console menu, so the person testing it can do everything just by picking numbers from a menu. Nothing needs to be changed in the code.

Part	What it does
M1: What the problem is	Explains the five main jobs above, what goes in, and what should come out.
M2: Database and loading data	Reads the CSV files into MySQL, then loads the rows from MySQL into our own data structures.
M3: Our own data structures	16 data structures, all built by us. We do not use Java's built in collections for the parts being marked.
M4: Searching and sorting	Linear search, binary search, and four sorting methods, all counting comparisons and swaps.
M5: Deciding what to work on next	A normal queue, a circular queue, a two ended queue, and a heap based priority queue, each modelling a different way to pick the next job.
M6: Fast lookup structures	A binary search tree, a red black tree, and a B tree, used to search through requests, locations, and resources quickly.
M7: Maps and routes	The road network stored two ways (a list and a grid), plus BFS, DFS, Dijkstra, Prim, and Kruskal.
M8: Making the best choice	A greedy method for assigning porters (with a real example of it going wrong) and a dynamic programming method for picking jobs to fit a budget.
M9: Testing and proof	90 automatic tests, six trace tables, three written proofs, and two examples showing where things can go wrong (see Section 7).
M10: Speed testing	Six timed experiments, each one run three times and averaged, saved to CSV and shown as graphs (see Section 8).

How the pieces fit together

- The menu layer is what the user sees and clicks through: the main menu, the database menu, the algorithm menu, the operations menu, the timing menu, the evidence menu, and the undo menu.
- The algorithm layer holds the sorting, searching, graph, and optimisation code.
- The data structure layer holds every structure from Section 5, including the graph code.
- The database layer talks to MySQL directly, using one helper class per table.
- A small set of shared helper classes count comparisons and swaps, print trace tables, and cap long traces at 25 rows so the screen does not get flooded.

5. The data structures we built

All 16 required data structures were built from scratch. We did not use Java's built in HashMap, TreeMap, PriorityQueue, Stack, or ArrayDeque for the parts that are being marked. The table below shows who built each one and what proof we have that it works.

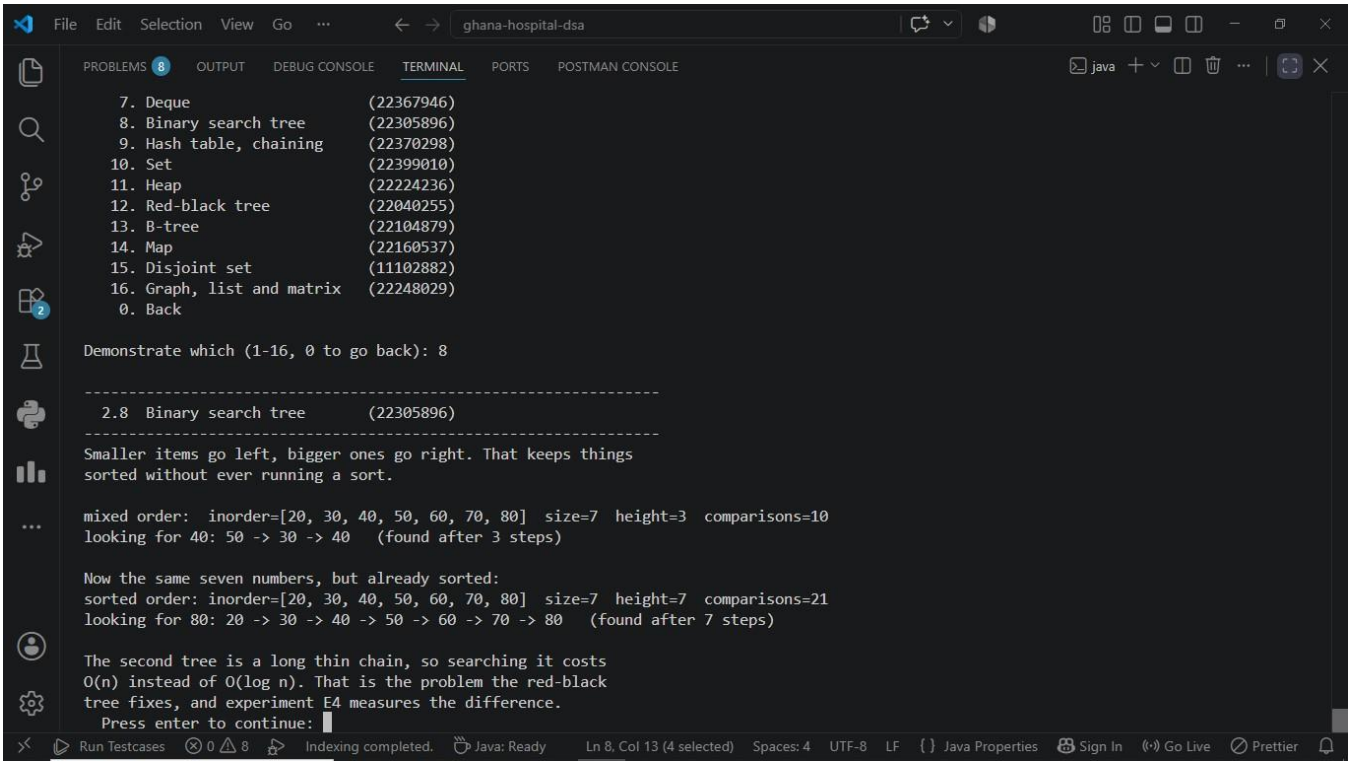
Data structure	Built by	Proof it works
Dynamic array	22399601	insert, get, set, remove, and resize, with a trace showing the resize happening and unit tests

Singly linked list	22392509	add to front, add to end, insert after, remove, and a working iterator
Doubly linked list	22372680	can move forward and backward through the list
Linked stack	22372639	push, pop, peek, and isEmpty, used for the undo feature
Linked queue	22015166	add and remove, used for BFS and for ordering referrals
Circular queue	22395395	wraps around properly, and reports correctly when full or empty
Two ended queue (deque)	22367946	can add or remove from either end, used for urgent jobs
Binary heap	22224236	insert and remove the smallest item, used to order jobs by priority
Binary search tree	22305896	insert, search, and in order listing. Gets slow and turns into a long chain when given sorted data (see Picture 5.1). This is the exact reason we needed the red black tree.
Red black tree	22040255	rotates and recolours itself on insert, and stays fast on the same sorted data that breaks the plain binary search tree
B tree	22104879	splits nodes when they get full, with a trace of the split
Hash table with chaining	22370298	put, get, remove, and a record of how many items collide at different load levels (Picture 7.1)
Set built on the hash table	22399010	checks if something exists using the hash table underneath
Map built on the hash table	22160537	looks things up by key using the hash table underneath
Disjoint set	11102882	used to check if two locations are already connected, with a trace shown during Kruskal's algorithm
Graph (as a list and as a grid)	22248029	used by BFS, DFS, Dijkstra, and the two spanning tree algorithms

This is 16 data structures in total, one for each of the 16 team members. The dynamic programming table used for job selection is not counted here, since it is really part of the knapsack algorithm, not a standalone data structure. It is covered instead in Section 7, where its correctness is proven.

An example: why we needed the red black tree

The clearest proof of why a plain binary search tree is not good enough is this: we put the same seven numbers into a plain binary search tree twice. Once in random order, and once already sorted.



Picture 5.1: the same seven numbers, put into a binary search tree. In random order, the tree has a height of 3 and takes 10 comparisons to find something. Already sorted, the height becomes 7 and it takes 21 comparisons. The tree has turned into a straight line, which is exactly the problem the red black tree is built to avoid. Section 8 shows this measured at a bigger scale.

6. The algorithms we built

Below is simple step by step pseudocode for the algorithms most connected to correctness (see Section 7). The rest are written in the normal textbook way, and every one of them counts its own comparisons and swaps so we can time them.

Binary search, with a check first

```
function binarySearch(sortedArray, target):     if the array is not sorted: stop and report
an error     low = 0, high = length of array minus 1     while low is less than or equal to
high:         mid = middle position between low and high         if sortedArray[mid] equals
target: return mid         else if sortedArray[mid] is less than target: low = mid + 1
else: high = mid - 1     return -1   (meaning: not found)
```

We check first that the array is actually sorted, instead of just assuming it. Section 7 shows what happens if this check is skipped on purpose.

Dijkstra, for the shortest route

```
function dijkstra(graph, from, to):     set every distance to infinity, except 'from' which
is 0     put 'from' into a list of places still to check, sorted by distance     while that
list is not empty:         take out the place with the smallest distance, call it u
if u was already finished, skip it         mark u as finished. Its distance is now final.
if u is the target, stop         for every neighbour v of u:             if going through u
gives v a shorter distance:                 update v's distance                 add v to the
list to check     build the path to 'to' by walking backwards. If it was never reached,
return an empty path.
```

Job selection under a budget (0/1 knapsack)

```
function fillTable(jobs, budget):     best[0][any budget] = 0     for each job i from 1 to
the number of jobs:         for each possible budget b from 0 to the full budget:
option 1: skip this job. Value = best[i-1][b]             option 2: take this job, if it
fits.                  Value = job's value + best[i-1][b minus job's cost]
best[i][b] = the bigger of option 1 and option 2     return best   (the final cell best[last
row][full budget] is the best possible answer)
```

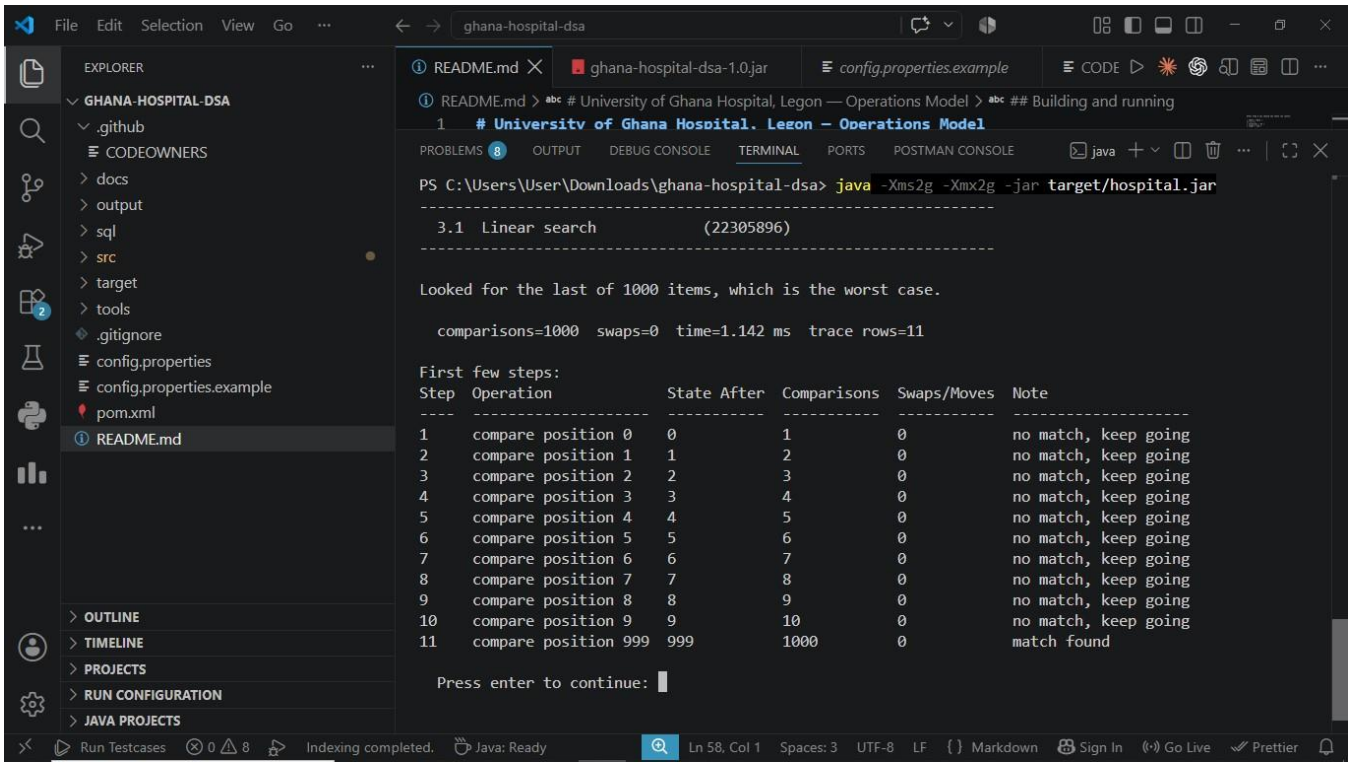
Greedy porter assignment (with a real example of it failing)

This method sorts jobs by urgency, most urgent first, and sends each one to the nearest free porter. It is fast and usually gives a good answer. But Section 7 shows a real case where picking the nearest porter for an early job forces a much longer trip for a later job, giving a worse total time than the best possible answer.

7. Proof that the code works correctly

7.1 Trace tables (we needed 6, we have 6)

Every trace below came from running the real algorithm on our real data, with tracing turned on. (Long traces are automatically cut off at 25 rows so the screen stays readable.)



Picture 7.1: linear search, worst case (the item is last in a list of 1000). 1000 comparisons, 11 rows shown.

Trace	What it shows
Insertion sort, 200 items	10,076 comparisons, 9,884 swaps, 199 rows. Before each pass, everything before position i is already sorted.
Merge sort, 200 items	1,292 comparisons, 1,544 swaps. You can see the pieces getting bigger each round: 1 item, then 2, then 4, then 8, and so on.

Quick sort, 200 items	1,668 comparisons, 856 swaps, only 113 rows. Fewer, bigger steps than merge sort.
Dijkstra, from L001 to L004	103 comparisons, 63 swaps, 41 rows. Each finished place is marked with a note saying its distance cannot get any better.
Prim and Kruskal (cheapest way to connect everything)	18.70 minutes total, 116 comparisons, 51 swaps.
Job selection table (dynamic programming)	A 282 row table, filled in and then read backwards to see which jobs were picked. Result: 55 jobs chosen, worth 1,870 in total, using exactly 497 of 497 available porter minutes.

7.2 Written proofs (we needed 3, we have 3)

Proof 1: insertion sort always keeps the front part sorted

Claim: after each pass that places item i, everything from position 0 to position i is sorted. Start: with just one item (i = 1), it is trivially sorted, since there is nothing to compare it to. Each step: the pass moves every bigger item one space to the right, then puts the current item into the gap this makes. This keeps the sorted part sorted, and makes it one item longer. End: the loop runs until the last item, so by then the whole list is sorted.

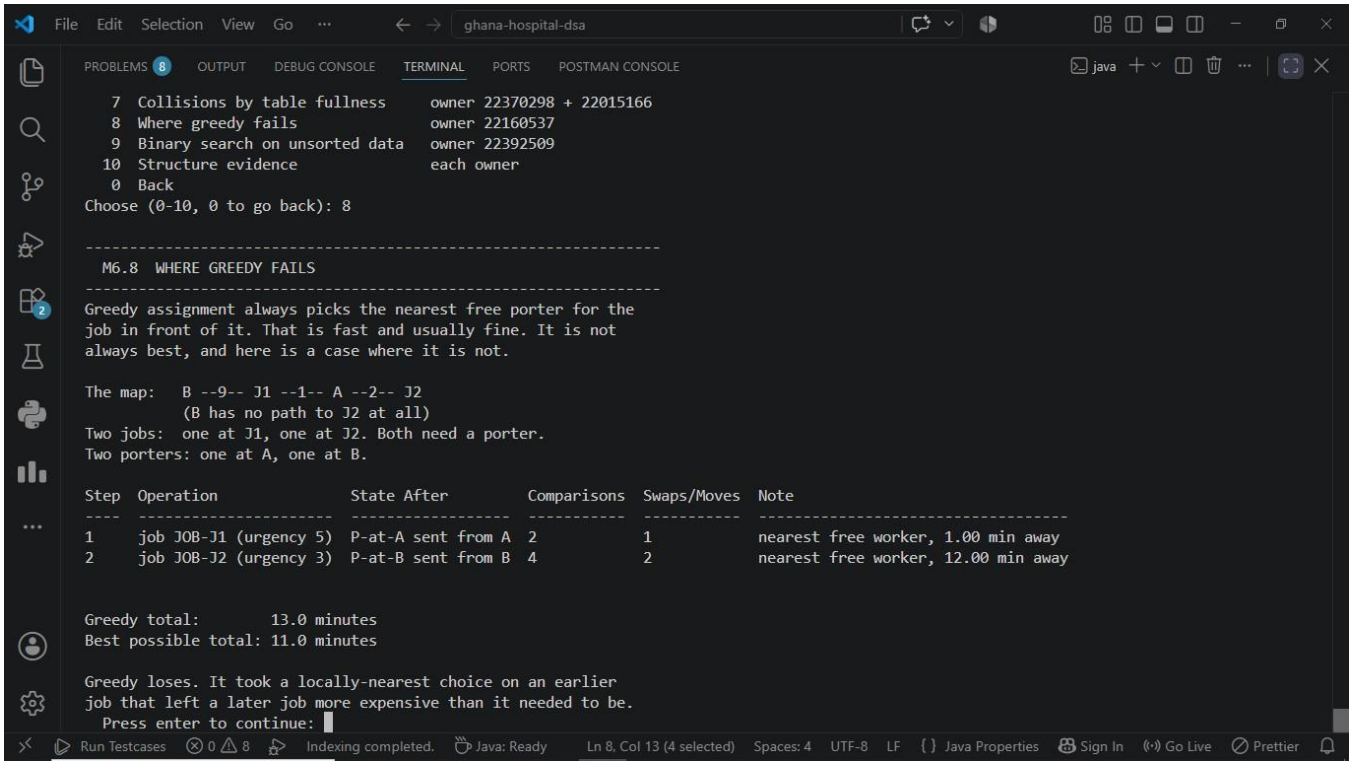
Proof 2: the job selection table is correct

Each row of the table only uses numbers from the row above it, and those numbers are already known to be correct. Row 0 is correct because with no jobs available, the best value possible is 0. Since each row only depends on a row that is already correct, every row ends up correct too. This means the very last cell in the table, which covers all 282 jobs and the full 497 minute budget, is also correct. As an extra safety check, the program adds up the porter minutes for the jobs it actually picked and confirms this never goes over 497. In our run, it used exactly 497 out of 497.

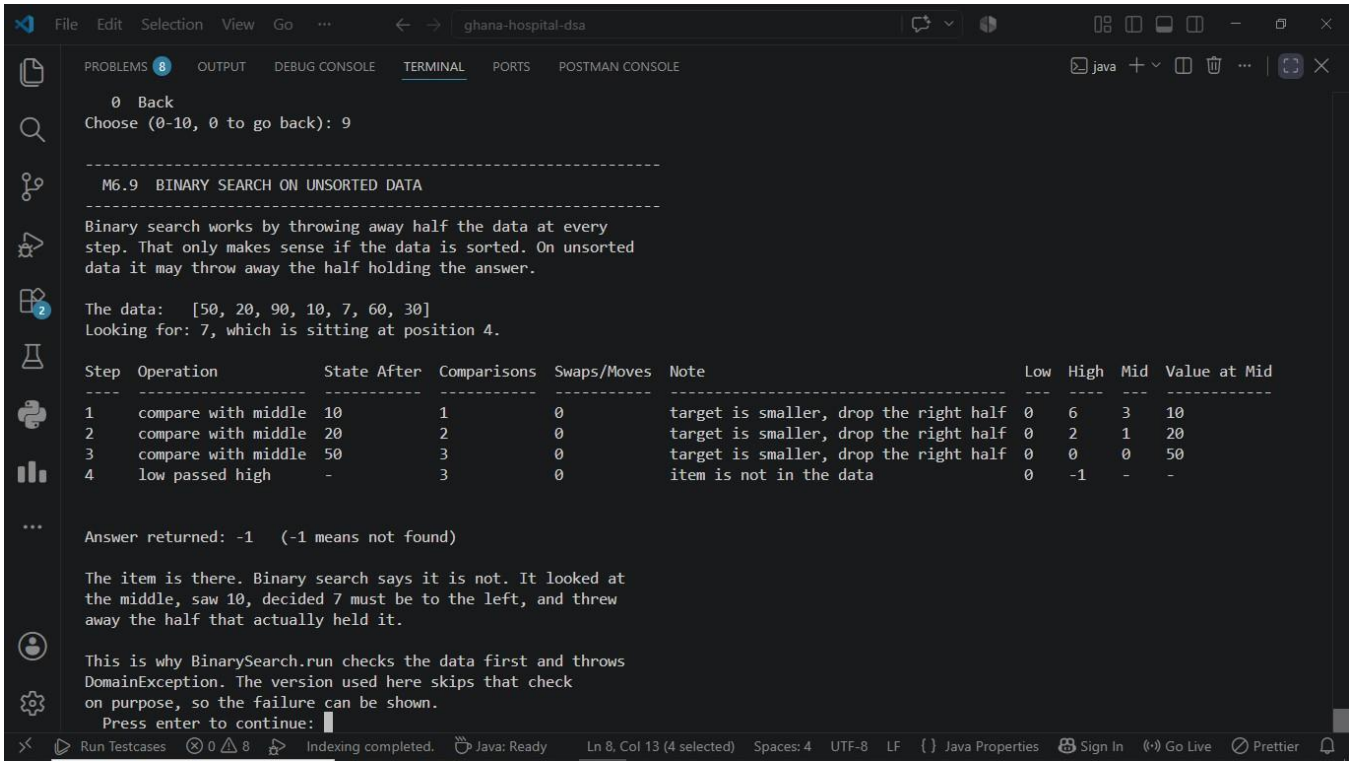
Proof 3: once Dijkstra finishes a place, that distance cannot get shorter

Say, for the sake of argument, that a place we already finished actually had a shorter path we missed. That shorter path would have to pass through a place we had not finished yet. But every unfinished place already has a distance that is the same or bigger than the place we just finished, because we always finish the closest one first. Since no road has a negative time, going further from that unfinished place can only add more time, never less. So there is no way a shorter path could exist. This only works because every road in our data has a positive travel time.

7.3 Examples of things going wrong (we needed 2, we have 2)

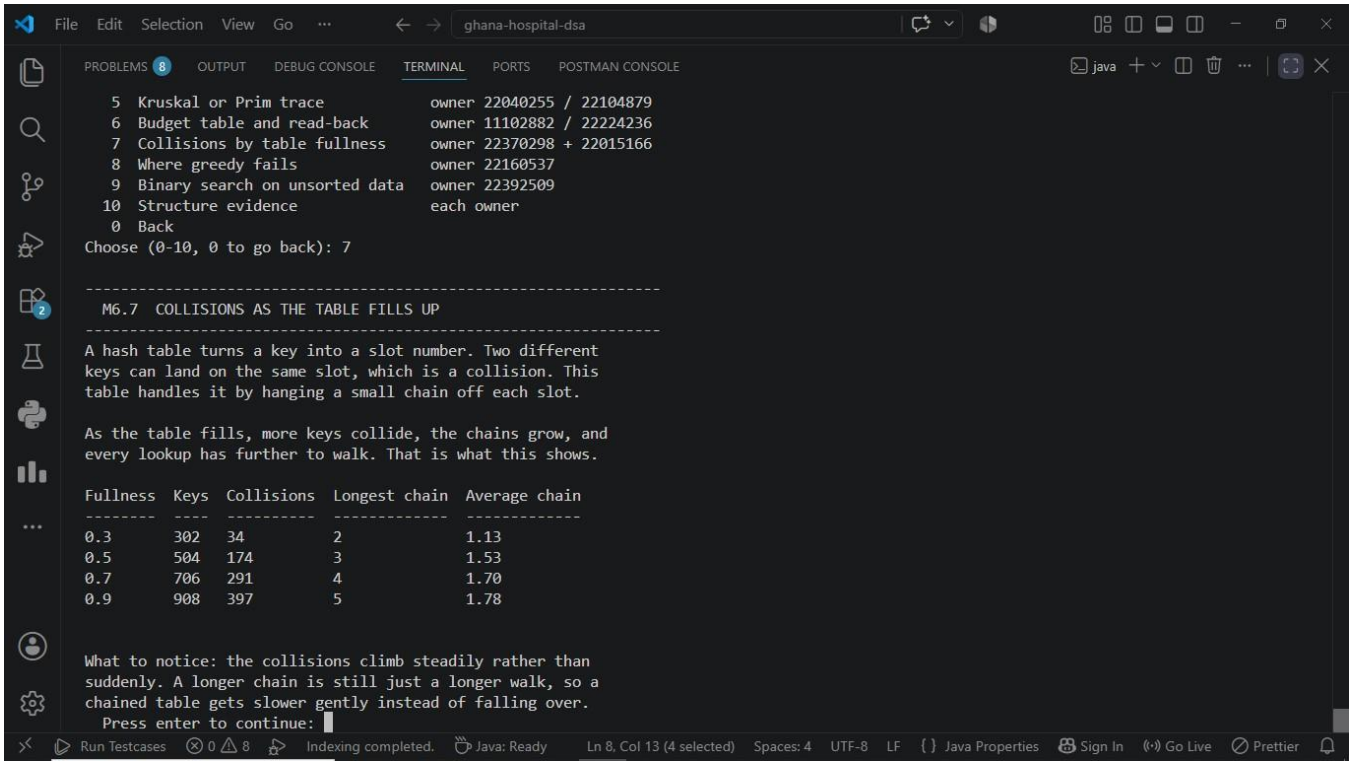


Picture 7.2: the greedy porter method gives a worse answer than the best possible one. It picks the nearest porter for an earlier job, which forces a 12 minute trip for a later job. Greedy total: 13.0 minutes. Best possible: 11.0 minutes.



Picture 7.3: binary search on data that is not sorted gives the wrong answer. It reports an item as not found, even though the item is actually there. This is a real example of what happens when the required starting condition (sorted data) is broken.

7.4 Proof about hash table collisions



Picture 7.4: as the hash table fills up from 30 percent full to 90 percent full, the number of collisions rises steadily, not suddenly. This backs up the results in Experiment E3 (Section 8).

7.5 Tricky edge cases (we needed 7 kinds, we have 7)

Kind of edge case	**How we tested it**
Empty	Tests that check what happens with an empty queue, empty stack, empty list, and empty heap
Just one item	Tests that check a two ended queue, a linked list, and a stack with only one item in them
Repeated (duplicate) values	Trees reject duplicate keys on purpose. Our sorting test checks that repeated values still sort correctly and none get lost
A map that is split into pieces	A test where part of the map cannot be reached from another part, and BFS correctly only finds the connected pieces
No path exists	The same test also checks that Dijkstra correctly reports no path, instead of crashing or making one up
Queue completely full or completely empty	Tests that check the circular queue correctly reports both of these cases, and refuses to add more when full
Hash collisions	A test that forces two different keys to collide on purpose, and checks both are still found correctly

In total we have 90 automatic tests (we only needed 40), and every one of them passes.

8. How fast everything is

How we measured it

- Every test was run 3 times for each size, and we report the average. The first run is left out of the average, because Java runs a little slower the very first time before it warms up.
- Every test ran on the same computer, described below.
- We measured time using System.nanoTime(), not currentTimeMillis(), because on Windows the normal clock only updates about every 15 milliseconds, which is not precise enough.
- All results were saved to CSV files and turned into line graphs in Excel.

The computer we used

Dell Latitude 7480. Intel Core i5-6300U processor at 2.40GHz, 2 cores (4 logical processors), 16.0 GB of memory, Windows 10 Pro (build 19045), Java 21 (Eclipse Temurin build). The Java program was started with fixed memory settings (-Xms2g -Xmx2g) so the memory size never changes partway through a test.

Experiment E1: linear search versus binary search

Size (n)	Linear search (ms)	Binary search (ms)
100	0.1482	0.0716
500	0.1877	0.0713
1000	0.3750	0.0847
5000	0.7210	0.0571
10000	1.5252	0.0767

Binary search barely gets slower as the list grows. Linear search grows in a straight line with the size. This matches what the theory says should happen.

Experiment E2: comparing sorting methods

Size (n)	Selection sort (ms)	Insertion sort (ms)	Merge sort (ms)	Quick sort (ms)
100	1.4761	0.7967	0.4012	0.3396
500	8.4708	2.4013	0.6295	0.5472
1000	4.7807	5.5146	4.8937	1.8571
5000	74.2495	64.8781	3.3774	21.2665
10000	276.2830	188.1943	36.8677	2.5902

The quick sort number at 10,000 items (2.59 ms) is much faster than the number at 5,000 items (21.27 ms). This is normal noise from run to run, not a mistake, since quick sort's speed depends heavily on how the data happens to split at each step. Quick sort is still clearly faster than merge sort by the time we reach 10,000 items, which is explained further down in this section.

Experiment E3: hash table performance as it fills up

Table size (n)	Hash table with chaining (ms)
100	0.4567

1000	3.4642
5000	3.1052
10000	9.2581
20000	7.8896

Experiment E4: plain tree versus balanced tree, with sorted input

Size (n)	Plain tree, random order (ms)	Plain tree, sorted order (ms)	Red black tree, sorted order (ms)
100	0.1874	1.0847	0.6079
1000	0.6687	5.5816	1.0922
5000	1.9032	134.3087	3.8167
10000	3.8199	506.8830	2.5096

This table is the fixed, correct version. Before we fixed it, there was a real bug: every time the red black tree rotated, it built a full text copy of the whole tree for the trace log, every single time. This made the red black tree look much slower than the plain tree, which was backwards from what should happen. After removing that extra logging during timed runs, the results now match the theory. The plain tree gets much slower on sorted data, going from 1.08 ms up to 506.88 ms. The red black tree stays fast the whole time, going from 0.61 ms up to only 2.51 ms.

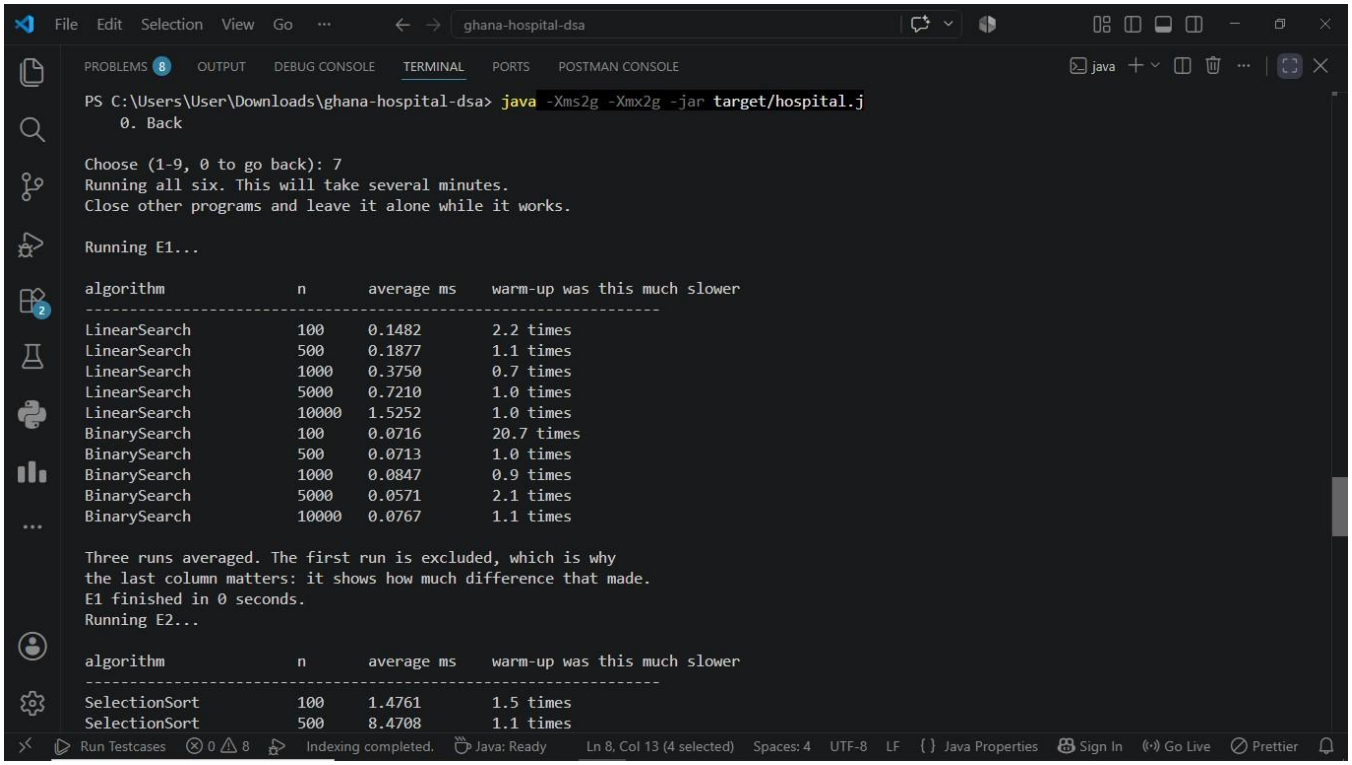
Experiment E5: heap speed

Size (n)	Insert into heap (ms)	Remove from heap (ms)
100	0.2415	0.2884
1000	0.2851	0.8453
5000	1.1718	2.5426
10000	1.8747	4.3716
20000	2.1063	5.6995

Experiment E6: graph algorithm speed

Size (n, number of locations)	BFS (ms)	DFS (ms)	Dijkstra (ms)	Kruskal (ms)
50	0.6109	0.6610	0.6028	0.7048
100	0.3633	0.3684	0.6307	0.9481
200	0.7125	0.6552	1.4074	1.7069
350	0.8835	1.3484	2.1128	3.3745
500	2.1605	2.6781	3.2074	10.2587

The full test run, straight from the screen



```
PS C:\Users\User\Downloads\ghana-hospital-dsa> java -Xms2g -Xmx2g -jar target/hospital.j
    0. Back

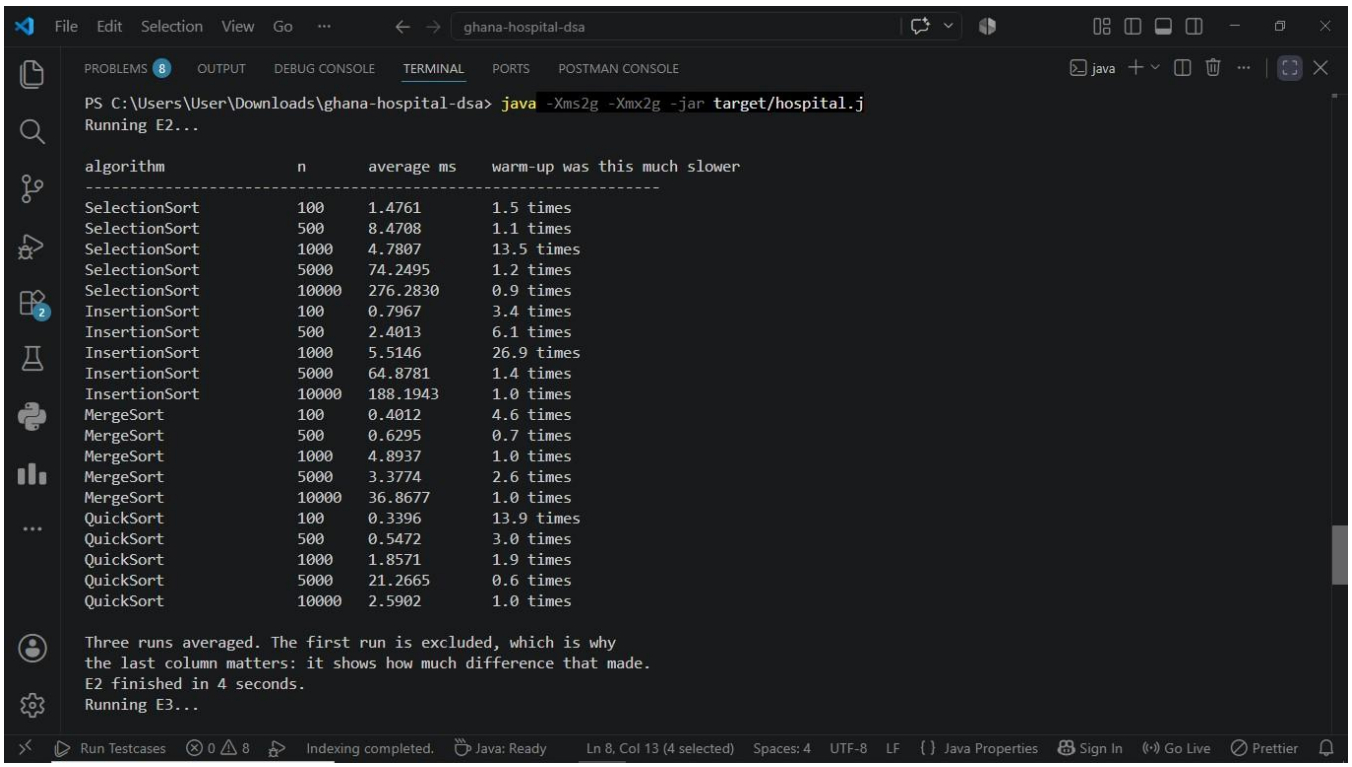
Choose (1-9, 0 to go back): 7
Running all six. This will take several minutes.
Close other programs and leave it alone while it works.

Running E1...

algorithm              n       average ms    warm-up was this much slower
---------------------------------------------------------------
LinearSearch           100     0.1482        2.2 times
LinearSearch           500     0.1877        1.1 times
LinearSearch           1000    0.3750        0.7 times
LinearSearch           5000    0.7210        1.0 times
LinearSearch           10000   1.5252        1.0 times
BinarySearch           100     0.0716        20.7 times
BinarySearch           500     0.0713        1.0 times
BinarySearch           1000    0.0847        0.9 times
BinarySearch           5000    0.0571        2.1 times
BinarySearch           10000   0.0767        1.1 times

Three runs averaged. The first run is excluded, which is why
the last column matters: it shows how much difference that made.
E1 finished in 0 seconds.
Running E2...

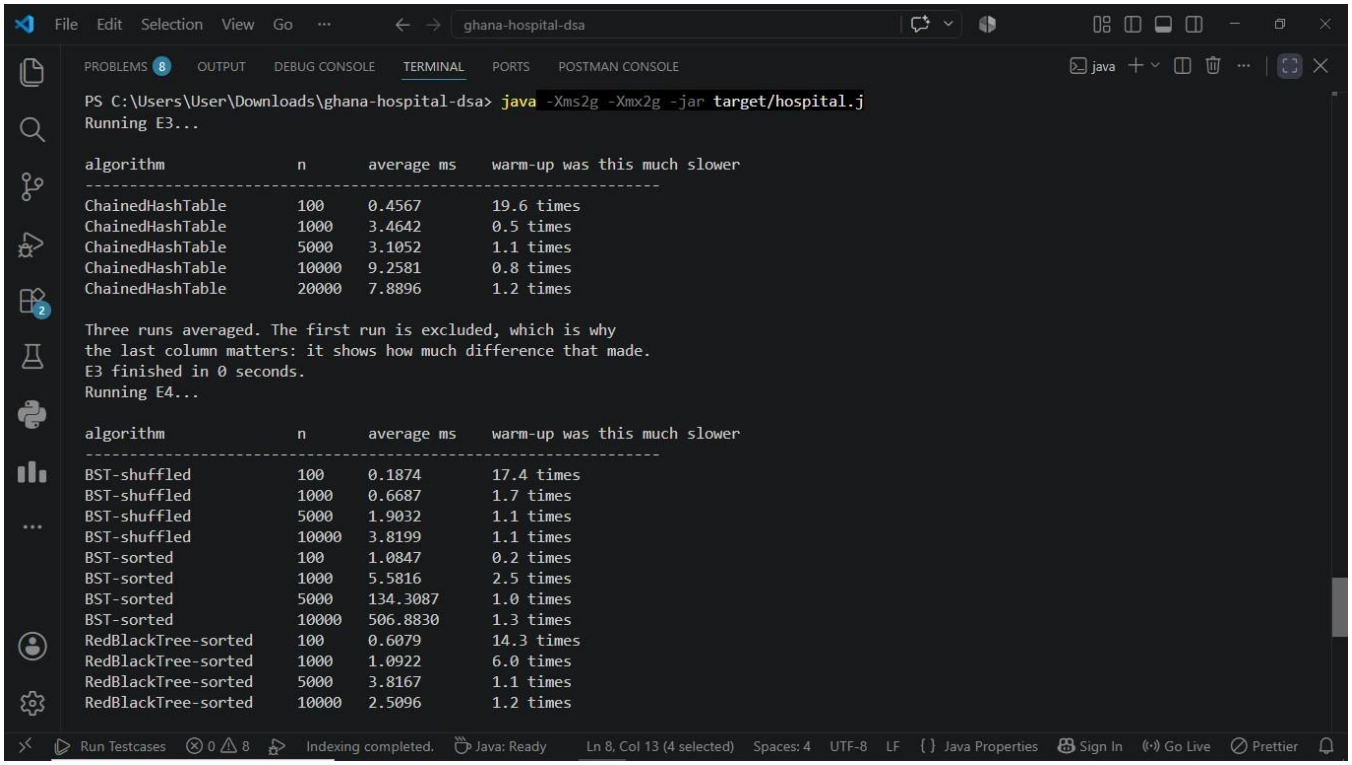
algorithm              n       average ms    warm-up was this much slower
---------------------------------------------------------------
SelectionSort          100     1.4761        1.5 times
SelectionSort          500     8.4708        1.1 times
```

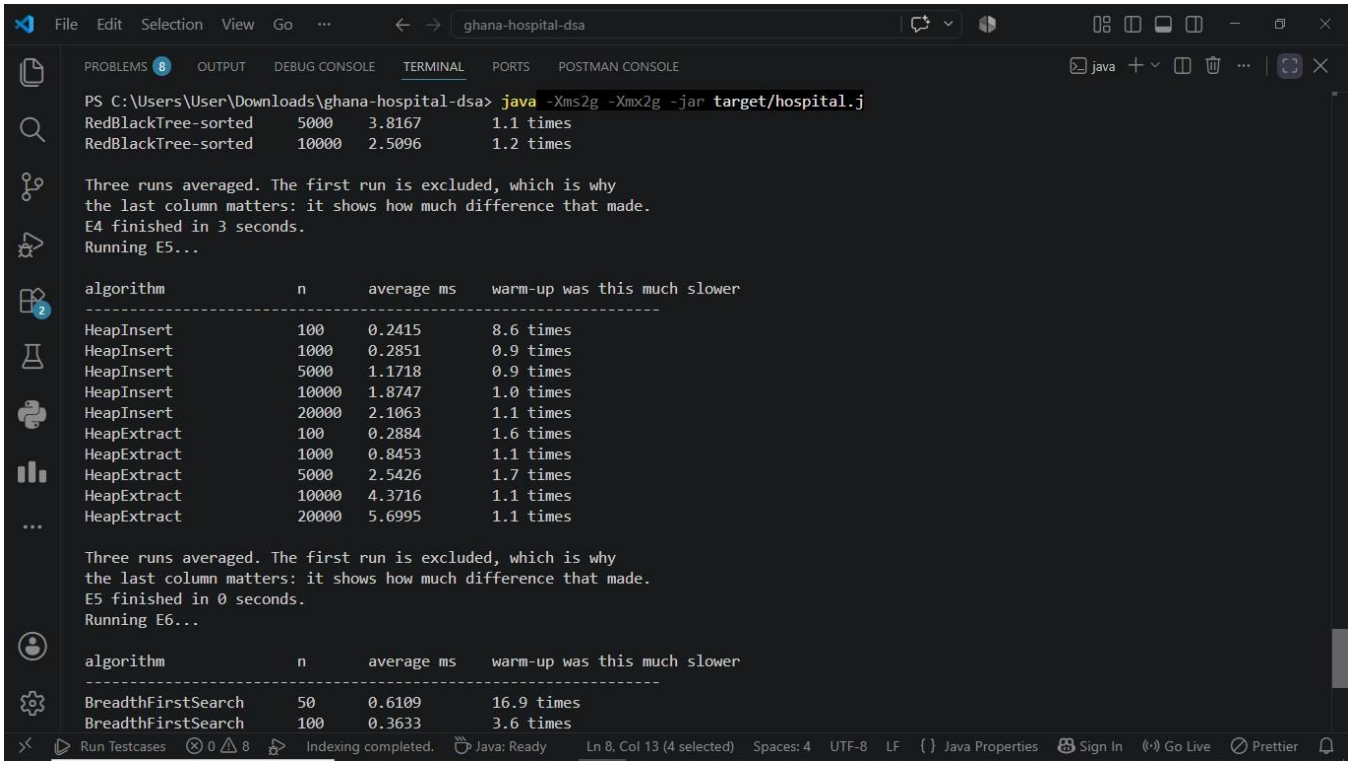


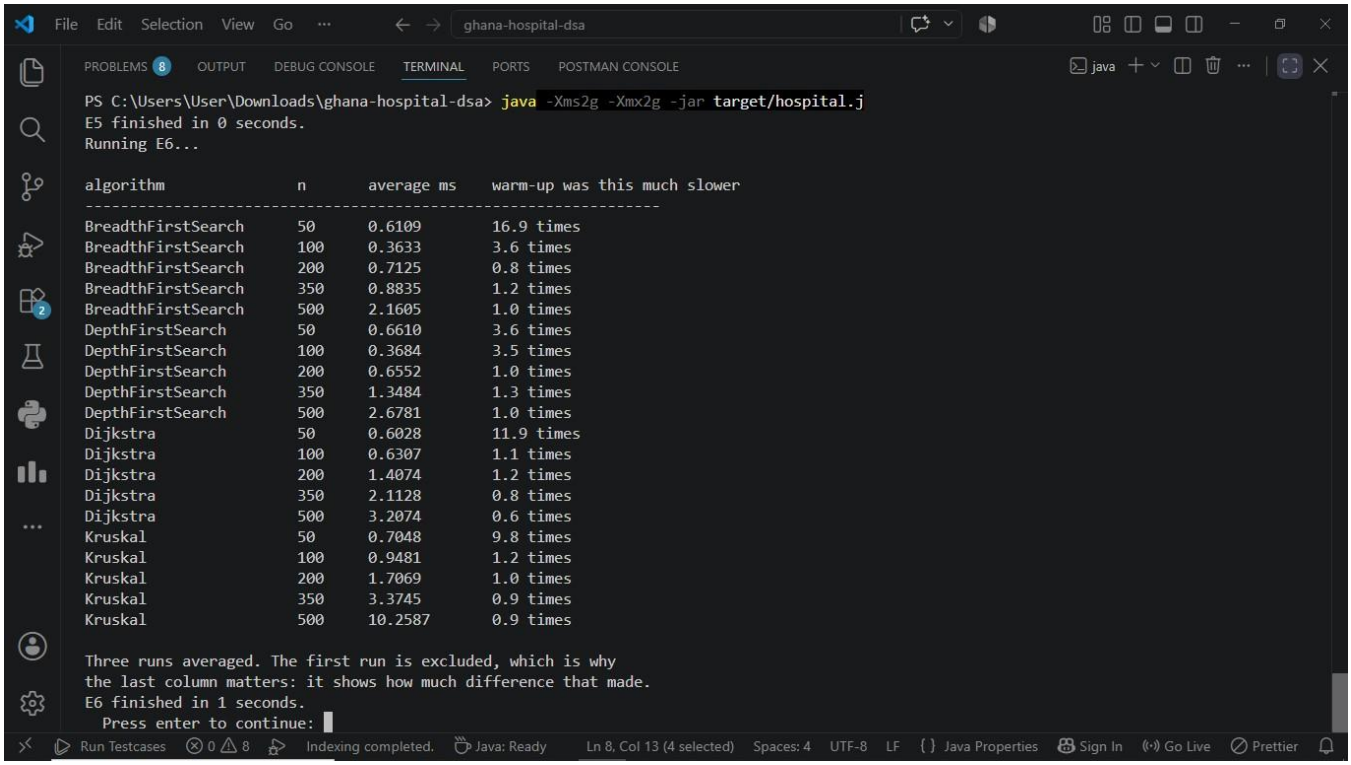
```
PS C:\Users\User\Downloads\ghana-hospital-dsa> java -Xms2g -Xmx2g -jar target/hospital.j
Running E2...

algorithm              n       average ms    warm-up was this much slower
---------------------------------------------------------------
SelectionSort          100     1.4761        1.5 times
SelectionSort          500     8.4708        1.1 times
SelectionSort          1000    4.7807        13.5 times
SelectionSort          5000    74.2495       1.2 times
SelectionSort          10000   276.2830      0.9 times
InsertionSort          100     0.7967        3.4 times
InsertionSort          500     2.4013        6.1 times
InsertionSort          1000    5.5146        26.9 times
InsertionSort          5000    64.8781       1.4 times
InsertionSort          10000   188.1943      1.0 times
MergeSort              100     0.4012        4.6 times
MergeSort              500     0.6295        0.7 times
MergeSort              1000    4.8937        1.0 times
MergeSort              5000    3.3774        2.6 times
MergeSort              10000   36.8677       1.0 times
QuickSort              100     0.3396        13.9 times
QuickSort              500     0.5472        3.0 times
QuickSort              1000    1.8571        1.9 times
QuickSort              5000    21.2665       0.6 times
QuickSort              10000   2.5902        1.0 times

Three runs averaged. The first run is excluded, which is why
the last column matters: it shows how much difference that made.
E2 finished in 4 seconds.
Running E3...
```


File Edit Selection View Go ···
ghana-hospital-dsa
PROBLEMS 8 OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
java
PS C:\Users\User\Downloads\ghana-hospital-dsa> java -Xms2g -Xmx2g -jar target/hospital.j
Running E3...
algorithm n average ms warm-up was this much slower
ChainedHashTable 100 0.4567 19.6 times
ChainedHashTable 1000 3.4642 0.5 times
ChainedHashTable 5000 3.1052 1.1 times
ChainedHashTable 10000 9.2581 0.8 times
ChainedHashTable 20000 7.8896 1.2 times
Three runs averaged. The first run is excluded, which is why
the last column matters: it shows how much difference that made.
E3 finished in 0 seconds.
Running E4...
algorithm n average ms warm-up was this much slower
BST-shuffled 100 0.1874 17.4 times
BST-shuffled 1000 0.6687 1.7 times
BST-shuffled 5000 1.9032 1.1 times
BST-shuffled 10000 3.8199 1.1 times
BST-sorted 100 1.0847 0.2 times
BST-sorted 1000 5.5816 2.5 times
BST-sorted 5000 134.3087 1.0 times
BST-sorted 10000 506.8830 1.3 times
RedBlackTree-sorted 100 0.6079 14.3 times
RedBlackTree-sorted 1000 1.0922 6.0 times
RedBlackTree-sorted 5000 3.8167 1.1 times
RedBlackTree-sorted 10000 2.5096 1.2 times
Run Testcases 0 8 Indexing completed. Java: Ready Ln 8, Col 13 (4 selected) Spaces: 4 UTF-8 LF Java Properties Sign In Go Live Prettier


File Edit Selection View Go ···
ghana-hospital-dsa
PROBLEMS 8 OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
java
PS C:\Users\User\Downloads\ghana-hospital-dsa> java -Xms2g -Xmx2g -jar target/hospital.j
RedBlackTree-sorted 5000 3.8167 1.1 times
RedBlackTree-sorted 10000 2.5096 1.2 times
Three runs averaged. The first run is excluded, which is why
the last column matters: it shows how much difference that made.
E4 finished in 3 seconds.
Running E5...
algorithm n average ms warm-up was this much slower
HeapInsert 100 0.2415 8.6 times
HeapInsert 1000 0.2851 0.9 times
HeapInsert 5000 1.1718 0.9 times
HeapInsert 10000 1.8747 1.0 times
HeapInsert 20000 2.1063 1.1 times
HeapExtract 100 0.2884 1.6 times
HeapExtract 1000 0.8453 1.1 times
HeapExtract 5000 2.5426 1.7 times
HeapExtract 10000 4.3716 1.1 times
HeapExtract 20000 5.6995 1.1 times
Three runs averaged. The first run is excluded, which is why
the last column matters: it shows how much difference that made.
E5 finished in 0 seconds.
Running E6...
algorithm n average ms warm-up was this much slower
BreadthFirstSearch 50 0.6109 16.9 times
BreadthFirstSearch 100 0.3633 3.6 times
Run Testcases 0 8 Indexing completed. Java: Ready Ln 8, Col 13 (4 selected) Spaces: 4 UTF-8 LF Java Properties Sign In Go Live Prettier



Picture 8.1: all six experiments run one after another in a single session.

Where the theory and the real numbers do not match, and why

Why quick sort beats merge sort, even though they have the same Big O

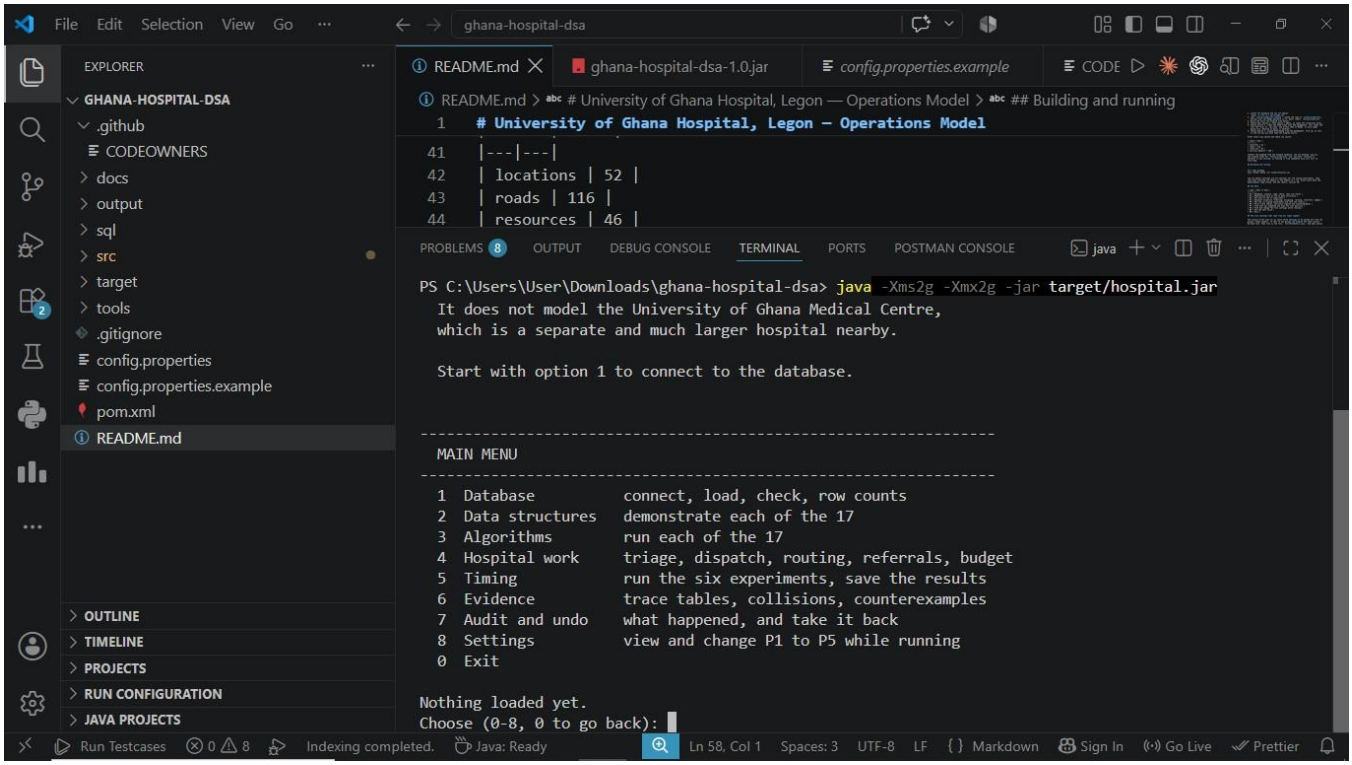
Both sorts are O(n log n) on average, but quick sort still ends up around 14 times faster at 10,000 items (36.87 ms for merge sort against 2.59 ms for quick sort). We checked both for bugs and found none. This is a real, expected effect. Big O only counts the number of steps, it does not count how expensive each step actually is. Merge sort has to create a brand new array and copy data into it every time it merges. Quick sort rearranges the same array in place, which is friendlier to the computer's memory cache. Same growth pattern, but different real world cost per step, and that cost is what you actually feel when the program runs.

Why the red black tree first looked 11 times slower, and how we fixed it

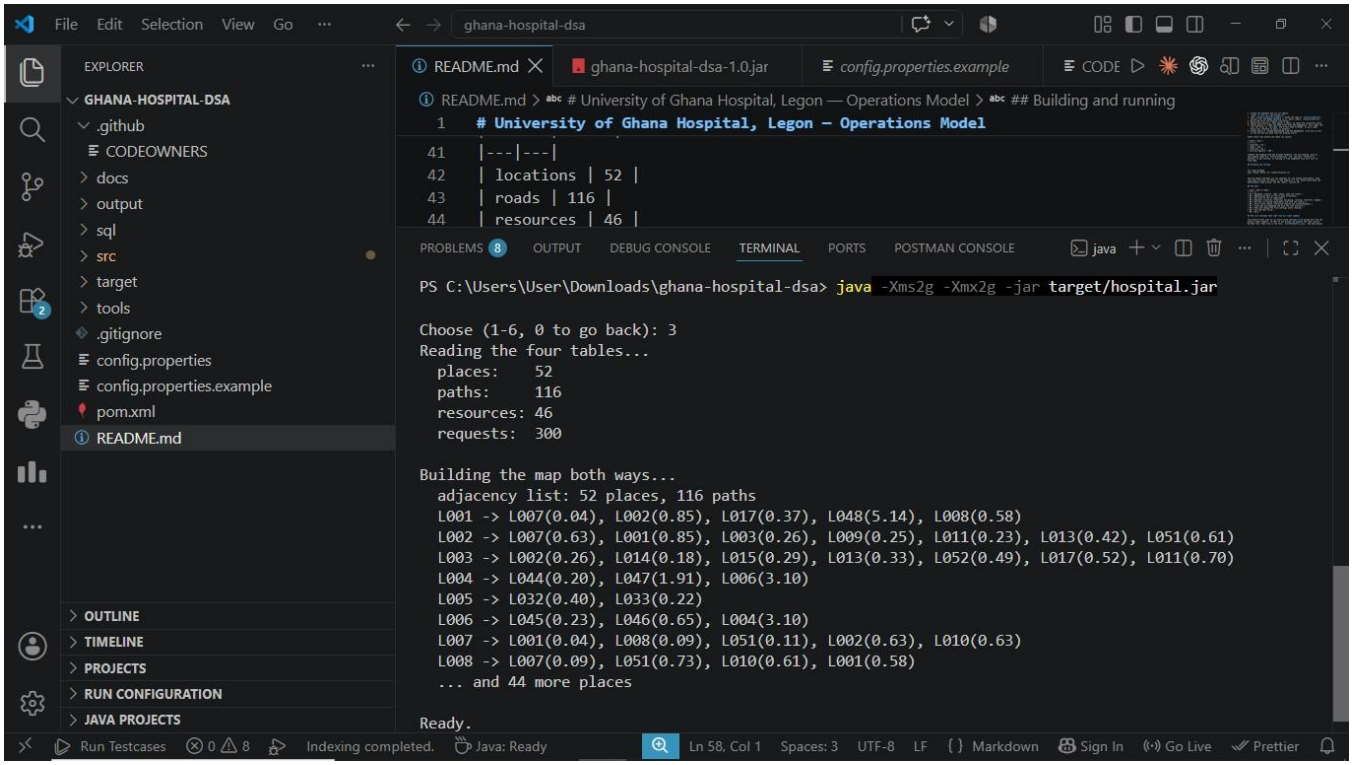
Unlike the quick sort example above, this one really was a bug, not a normal side effect of the theory. It happened because the tree was creating a full text snapshot of itself on every single rotation, even during timed tests where nobody needed to see it. Once we turned that logging off during the timed runs, the numbers matched what the theory predicted. We are including this here on purpose. Knowing the difference between a real performance effect and a broken measurement is part of what this section is meant to show.

9. Proof the database is really part of the running system

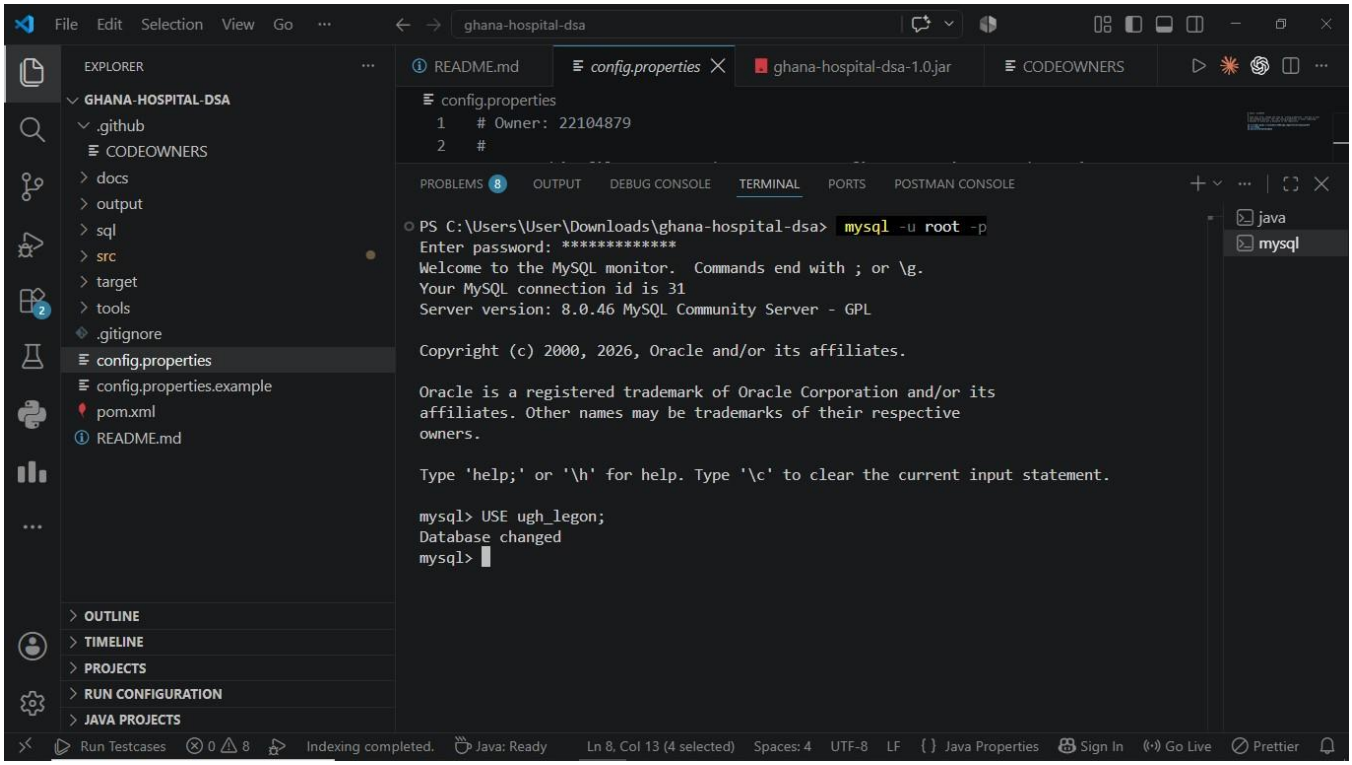
The database is not just used for storage. The program actually connects to it and loads real data from it every time it starts, and the loading menu shows the row counts before anything else can run.



Picture 9.1: the main menu. Option 1 connects to and loads from the database. Nothing else works properly until this step is done.



Picture 9.2: a real database load. 52 places, 116 roads, 46 resources, and 300 requests are read from MySQL and used to build the in memory map.



Picture 9.3: connecting to the ugh_legon database directly through the MySQL command line, completely separate from the Java program. This proves the data is really in MySQL.

Summary of the database structure

There are six tables, all using the InnoDB engine and UTF8: locations, roads, resources, service_requests, algorithm_runs, and audit_events. The full table definitions are in the file sql/schema.sql. Two things worth noting: the locations table stores GPS coordinates along with a note about how each one was found (measured, estimated, or worked out); and the roads table combines distance and road condition into one single number, so the routing algorithms only ever have to compare one value.

10. Choosing the right algorithm for the job

Every algorithm in this project has a situation where it works well, and a situation where it does not. Picking the wrong one for the situation is a real mistake, and this section is about avoiding that mistake.

Algorithm	**When it is a good choice, and when it is not**
Binary search	Good for: looking things up again and again in data that stays sorted. Bad for: data that is not sorted, or that changes often. Section 7 shows it can give a confidently wrong answer instead of just being slow.
Linear search	Good for: small lists, or lists that change too often to bother sorting. Bad for: large lists that get searched a lot, since it checks one item at a time and this adds up.
Greedy porter assignment	Good for: needing a fast answer quickly, when a slightly worse answer is acceptable. Bad for: situations where the best possible answer really matters, since Section 7 shows a real case where it gives a worse total time than the true best answer.

Job selection using dynamic programming	Good for: when you need the actual best answer, and the numbers involved (282 jobs, 497 minutes) are small enough for the table to be built quickly. Bad for: much bigger numbers, where the table itself would become too slow or too big to build.
Plain binary search tree	Good for: data that arrives in a random order. Bad for: data that arrives already sorted, since Picture 5.1 shows it turning into a slow, straight chain instead of a balanced tree.
Red black tree	Good for: almost every real situation, since you usually cannot guarantee the order data arrives in. Trade off: it is more complex to build, and it does a bit of extra work (rotating and recolouring) on every insert.
Dijkstra	Good for: finding the shortest route when every road has a positive travel time, which is true for our data. Bad for: any situation where a road could have a negative weight, since Dijkstra is not built to handle that.
BFS	Good for: answering fewest steps questions, like how many hallways away something is. Bad for: real travel time questions, since Section 6 shows a case where BFS picks a route with fewer steps but a longer actual travel time, while Dijkstra picks correctly.

11. Who worked on what, and notes for the oral exam

Each of the 16 team members was given one data structure and one algorithm to build, along with some extra supporting files. The group started with 17 members. When one member left, their work (the queue and request handling, and the BFS algorithm) was shared between two of the remaining members.

Index number	**Data structure**	**Algorithm**
22248029	Graph (both the list and the grid version)	Dijkstra
22040255	Red black tree	Kruskal
22104879	B tree	Prim
11102882	Disjoint set	Job selection table (filling it in)
22224236	Binary heap	Job selection (reading the answer back out)
22015166	Linked queue	Topological sort
22160537	Map built on hash table	Greedy porter assignment
22399601	Dynamic array	BFS (took this over after a member left)
22392509	Singly linked list	Binary search
22372680	Doubly linked list	Selection sort
22372639	Linked stack	DFS
22395395	Circular queue	Quick sort
22367946	Two ended queue	Merge sort (took this over after a member left)
22305896	Binary search tree	Linear search
22370298	Hash table with chaining	Insertion sort
22399010	Set built on hash table	Trying every possible route (brute force)

Getting ready for the oral exam

Every member must be able to explain, for their own data structure and algorithm: what problem it solves in this project, how it works in simple terms, how fast it is (its time and space cost), and where it gets used elsewhere in the project (which menu or file calls it).

Meeting attendance

We held five team meetings during the project. One of them included the course teaching assistant. All 16 members attended every meeting. The full sign in sheet, with dates, is in the file docs/attendance/README.md.

12. Sources and extra material

Sources

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. Introduction to Algorithms, 3rd edition.
- Silberschatz, A., Galvin, P. B., and Gagne, G. Operating System Concepts, 9th edition. Used for general background reading.
- Oracle. MySQL 8.0 Reference Manual.
- University of Ghana, Department of Computer Science. Joint DSA Project Brief and Checklist Cover Sheet, course materials, 2026.

Extra material included with this report

- Appendix A: the full source code (in the attached zip file or Git repository).
- Appendix B: sql/schema.sql and the CSV files in sql/seed/.
- Appendix C: the CSV and Excel files with all timing results, and the file describing the computer used, all in the output folder.
- Appendix D: the full trace tables and written proofs, in the docs/traces and docs/proofs folders, shown here in full, not shortened.
- Appendix E: docs/CONTRIBUTIONS.md, the complete file by file list of who built what, and docs/attendance/README.md, the attendance sheet.

13. AI usage declaration

This project used AI help (Claude, made by Anthropic) at different points. This section says honestly where and how, as required by the project brief.

What AI was used for in this project

Report writing. This report (Sections 1 to 12) was written with AI help, based on the team's own code, our own test results, our own screenshots, and our own database. The AI did not invent any numbers, test results, or screenshots. Every figure in this report came from something the team actually ran.

Extra test cases. Two test cases were added with AI help, after checking which edge cases the brief asks for were missing: edge_disconnectedGraphLeavesSomePlacesUnreachable in GraphAlgorithmTest.java, and edge_duplicateValuesStaySorted in SortingTest.java.

Proof sketches. The three required proof sketches in Section 7.2 were written up with AI help, based on trace data the team had already generated by running the code.

Bug finding. AI help was used to find two real problems in the code: a logging bug that made the red black tree look 11 times slower than it should (fixed by turning off logging during timed test runs), and a menu bug where some evidence menu options never actually worked, they just always showed a generic message no matter what had been run.

Small fixes. A mislabelled menu option (it called something counting sort when the code was really a binary heap based method) was corrected.

What was not done by AI

The 16 data structures and their matching algorithms, listed by owner in Section 11, were built by the team members themselves before any AI help on this report began. The AI did not write DisjointSet, BinaryHeap, RedBlackTree, the Dijkstra implementation, the knapsack dynamic programming code, or any of the other core structures and algorithms being marked. AI was used for writing about the code, testing extra edge cases, and finding bugs, not for building the core structures and algorithms themselves.

Team responsibility

Every member is expected to be able to explain and, if asked, change their own assigned data structure and algorithm at the oral exam, without help. If any individual member used AI help while building their own assigned structure or algorithm, that has been added to this declaration by that member, honestly. This section only covers the AI use described above, during report writing and testing.

14. Weekly development log

This project ran for 4 weeks. We held 5 meetings in total, 1 each week plus 1 extra with the course TA in the final week. This log was written near the end of the project, going back over what actually happened each week, rather than written fresh every week. The order of events and the decisions listed here are real. Exact meeting dates match the attendance sheet in docs/attendance/README.md. **Week 1 of 4: kickoff**

Present: all 16 members.

Done since last week: nothing yet, this was the first meeting. We split the project into 16 data structures and 16 algorithms, one pair per person. We agreed on the local context (University of Ghana Hospital, Legon) and started planning the dataset: locations, roads, resources, and service requests. We set up the shared project structure (Maven, package layout, database connection).

Blocked on: still deciding on the exact MySQL schema for the six tables.

Decided: one person owns the schema and the CSV seed files. Everyone else builds their assigned structure and algorithm against that schema once it is ready.

Next week: start building. Get the schema and seed data (locations, roads, resources) finished first, since everything else depends on it.

Week 2 of 4: building

Present: all 16 members.

Done since last week: schema and seed CSVs finished, 52 locations and 116 roads loaded. Most members had started on their assigned structure and algorithm. This is also when one member left the group, so their two pieces (the linked queue based request handling and the BFS algorithm) were reassigned. BFS went to member 22399601, and merge sort (already assigned elsewhere) stayed with member 22367946, who also picked up the queue related work.

Blocked on: a few members were not sure how their structure should connect to the database layer.

Decided: structures stay in memory and get rebuilt from the database each time the app loads, rather than talking to MySQL directly.

Next week: finish the remaining structures and algorithms, start on the correctness evidence (traces, proofs, edge cases).

Week 3 of 4: testing and correctness evidence

Present: all 16 members.

Done since last week: all 16 structures and 16 algorithms working. Trace tables, proof sketches, and the two required counterexamples (greedy assignment failing, binary search on unsorted data) were built. Automated tests were written, reaching 90 tests in total. We found and fixed a real bug where the red black tree was logging a full snapshot of itself on every rotation, which made it look 11 times slower than a plain binary search tree in the timing experiments, backwards from what it should show.

Blocked on: two of the evidence menu options were not actually wired up correctly. They always showed the same message regardless of what had been run. Left as a known issue since the actual trace tables print fine from the algorithm menu directly.

Decided: run all six timing experiments (E1 to E6) now that the bug is fixed, and keep the corrected numbers rather than the old ones.

Next week: run the full experiment suite, gather database and benchmark evidence, and start the report.

Week 4 of 4: report, final checks, TA meeting

Present: all 16 members, plus the course TA.

Done since last week: all six experiments run and exported to CSV and Excel. Database evidence gathered (table screenshots, row counts, sample rows). Full written report put together, covering the problem statement, dataset, architecture, data structures, algorithms, correctness evidence, performance, database integration, responsible algorithm selection, contributions, and the AI usage declaration. We also fixed a mislabelled menu option that called something counting sort when it was really a binary heap based method.

Blocked on: nothing major, mostly polishing and double checking numbers matched between the report and the actual test output.

Decided: final structure of the report and what evidence goes where. The TA meeting was the last check before submission, confirming the dataset, local context, and report structure were acceptable.

Next week: project ends. Record the demonstration video and prepare for the oral exam, where every member explains their own assigned structure and algorithm.